

Agentic Artificial Intelligence

Chapter 05: Agentic AI Engineering

From Architecture to Running Code

Prof. Dr. Thomas Clemen

BT7, R. 11.83 | thomas.clemen@haw-hamburg.de



Review + Questions

Semester Plan

Chapter 1. Agents and their Fundamentals

Chapter 2. Perception, Memory, Planning & Learning

Chapter 3. LLMs, RAG & Tool Use

Chapter 4. Multi-Agent Systems

Chapter 5. Agentic AI Engineering

Chapter 6. Safety & Guardrails

The Implementation Gap

We now know what a multi-agent system looks like. But how do we open a new Python file and start?



Which framework?

LangGraph, AutoGen, CrewAI – all can do it, but with fundamentally different assumptions about state and communication.



How to manage state?

The context window is no longer enough. State across multiple agents must be explicitly modeled.



How to test?

When the same input can produce different outputs, assertEquals no longer works. Testing is its own discipline.

From Architecture to Code

Chapter 04 defined the concepts – Chapter 05 provides the tools to implement them.

Chapter	Question	Focus
Ch. 01	What are the layers?	Architecture as analysis framework
Ch. 02	How do they work?	Mechanisms, types, limits
Ch. 03	How to realize with LLMs?	RAG = Memory, Tool Use = Action, ReAct = Planning
Ch. 04	How to coordinate many?	Architectures, communication, orchestration
Ch. 05	How to build in practice?	Frameworks, state management, testing
Ch. 06	How to keep it safe?	Safety, guardrails, risks

PART 1

Framework Landscape

LangGraph, AutoGen, CrewAI – three opinions on how to build agentic systems



Three Frameworks, Three Opinions

All three implement the same conceptual foundation – but with different assumptions.



LangGraph

Explicit State Graphs

Low-level control. State is typed and explicit. Nodes = functions/agents. Edges = transitions. Best for complex, stateful workflows.



AutoGen

Conversation-First

Agents are conversational partners. The orchestrator "talks" to workers via messages. Flexible and dynamic, harder to control.



CrewAI

Declarative Roles

Define crews with roles, goals, and tasks. Low boilerplate. Fast start for clearly structured, role-based pipelines.

Framework Comparison

Dimension	LangGraph	AutoGen	CrewAI
Abstraction Level	Low (explicit)	Medium	High (declarative)
State Model	Typed State Graph	Conversation history	Shared task objects
Communication	Shared State (default)	Message Passing	Hybrid
Control	Programmatic (code)	Conversation-based	Role-based
Debugging	Excellent (graph viz.)	Medium	Limited
Entry Barrier	Higher	Medium	Low
Best For	Complex stateful workflows	Dynamic agent conversation	Quick role-based start
Ch. 04 Mapping	Shared State + explicit decomp.	Message Passing + dynamic	Orchestrator/Worker decl.

LangGraph: Core Concepts

LangGraph makes architecture decisions explicit and visible — the topology of the graph IS the design.



Node

A function or agent (Orchestrator, Worker, Tool call).
Each node reads state, does work, writes state.



State

A shared, typed Python dict (TypedDict). All nodes read and write to it. Explicit, inspectable, type-safe.



Edge

A transition between nodes. Can be conditional: if result == 'fail' → retry_node.



Conditional Edge

Routing logic based on state content. The programmatic implementation of Replanning from Ch. 04.

LangGraph: Travel Agent as State Graph

The Orchestrator/Worker design from Ch. 04 maps directly onto a LangGraph State Graph.

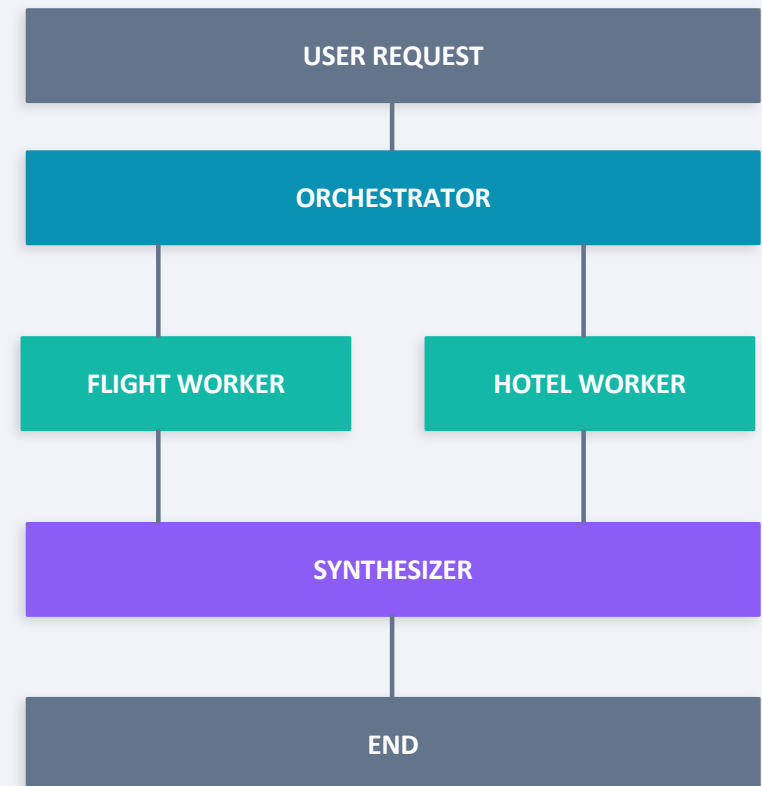
```
from langgraph.graph import StateGraph, END
from typing import TypedDict

class TravelState(TypedDict):
    user_request: str
    flights: list
    hotels: list
    errors: list

graph = StateGraph(TravelState)

graph.add_node("orchestrator", orchestrator)
graph.add_node("flight_worker", flight_agent)
graph.add_node("hotel_worker", hotel_agent)

graph.set_entry_point("orchestrator")
graph.add_conditional_edges(...)
app = graph.compile()
```



AutoGen: Conversation-First

Agents are conversational partners. The orchestrator "talks" to workers via structured messages.



Dynamic Interaction

No fixed graph — the orchestrator decides next steps through conversation. Highly flexible for open-ended tasks.



Message Passing Native

Each agent receives and responds to messages. Maps directly to the Message Passing pattern from Ch. 04.



Human-in-the-Loop Ready

Human agents can participate in the conversation. One human can interrupt, redirect, or approve.

✓ Use AutoGen when ...

- Task requires open-ended agent dialogue
- Dynamic replanning is more important than auditability
- Rapid prototyping of multi-agent conversations

✗ Avoid AutoGen when ...

- Strict, reproducible execution order required
- Deep debugging / tracing needed
- State must be fully typed and inspectable

CrewAI: Declarative Roles

Define agents through roles, goals, and tasks — minimal boilerplate, fast to get started.

```
from crewai import Agent, Task, Crew

flight_agent = Agent(
    role="Flight Specialist",
    goal="Find cheapest flight",
    backstory="Expert in flight search"
)

flight_task = Task(
    description="Search HAM→NRT Feb 28",
    agent=flight_agent
)

crew = Crew(
    agents=[flight_agent, hotel_agent],
    tasks=[flight_task, hotel_task]
)
crew.kickoff()
```

✓ CrewAI strengths

Fast start · Role clarity · Low boilerplate · Good for demos

✗ CrewAI limitations

Less control over execution flow · Limited observability · Hard to debug complex state

Best use case

Clearly structured workflows where roles are stable and the primary goal is delivering results quickly — not analyzing internal behavior.

When to Use Which Framework?

The framework choice is an architecture decision — not a technical preference.

Need explicit control over execution flow
and typed, inspectable state?

→ LangGraph

Need dynamic, conversation-based agent
interaction and flexible replanning?

→ AutoGen

Want fast start with role-based pipeline
and minimal boilerplate?

→ CrewAI

⚠ All three frameworks can implement the same architecture — the difference is how explicit and observable the implementation is.

PART 2

State Management

How agents share, persist, and recover their knowledge



The Problem: Whose Memory Is It?

In a single-agent system, memory = context window. In multi-agent systems, it gets complicated.

Orchestrator

Context Window:

Sees: user request, all results, plan

Flight Worker

Context Window:

Sees: subtask only, flight results

Hotel Worker

Context Window:

Sees: subtask only, hotel results

No shared memory

Workers don't see each other's results unless explicitly passed.

Consistency risk

Hotel Worker may act on outdated flight info if state is not synchronized.

Replanning requires full state

Orchestrator can only replan if it has the complete picture — not just its own context.

Three Levels of State

Explicit state management is the foundation of a reliable, debuggable, replan-capable agentic system.

#	Level	What	Framework	Scope
1	Agent-Local State	Context window of each individual agent	LLM conversation history	Single agent, single run
2	Shared State	Common data store readable and writable by all agents	LangGraph State Dict / Shared Memory	All agents, current run
3	Persisted State	State that survives session boundaries and system restarts	Checkpoints, database	All agents, across runs

Design Rule:

Always make state explicit at Level 2 (Shared State) for any multi-agent workflow. Implicit state — scattered across agent context windows — is the primary cause of consistency bugs and untraceable failures.

State Graphs and Typed State

TypedDict state makes the contract between agents explicit — every field has a name, type, and owner.

```
from typing import TypedDict, List, Optional

class TravelState(TypedDict):
    # Input
    user_request: str

    # Worker results
    flights: List[dict]
    hotels: List[dict]
    meetings: List[dict]

    # Control flow
    errors: List[str]
    retries: int
    status: Optional[str] # "ok" | "partial" | "fail"
```

Explicit Contract

Every agent knows exactly what fields exist and what types they carry. No surprises.

Type Safety

Python type checkers catch mismatches before runtime. Wrong type = caught early.

Full Inspectability

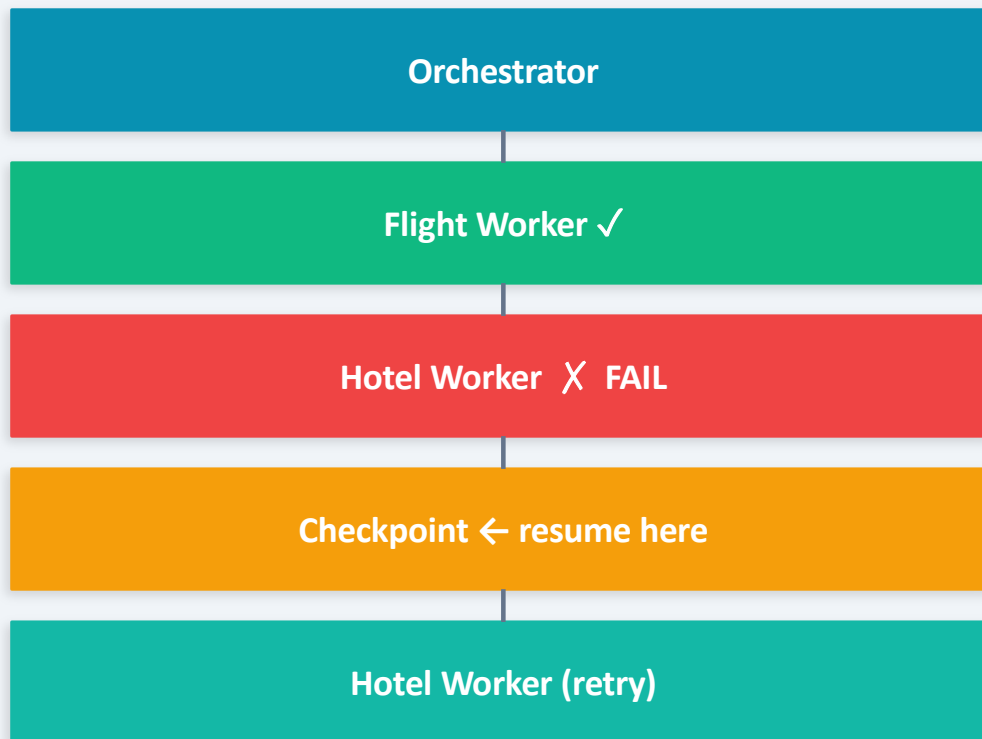
At any point, print the state dict and see the complete system snapshot.

Debuggable

When something goes wrong, the state is the log. Narrow down which field is wrong.

Checkpoints: Time Travel for Agents

Checkpoints save the complete state graph after each node. On failure: resume from the last valid snapshot.



```
from langgraph.checkpoint.memory import MemorySaver

# Create checkpointer
checkpointer = MemorySaver()
app = graph.compile(
    checkpointer=checkpointer
)

# Thread ID = persistent session
config = {"configurable": {
    "thread_id": "booking_42"
}}

# Run - checkpoints automatically saved
result = app.invoke(
    {"user_request": "..."},
    config=config
)
```

Memory Types Revisited

The four memory types from Ch. 02 reappear in frameworks — under different names.

Ch. 02 Type	Framework Equivalent	Example (Travel Agent)
Working Memory	State Dict (current run)	Current flights, hotel results, errors in the state graph
Long-Term Semantic Memory	Vector Store (RAG, Ch. 03)	Knowledge base of destinations, visa rules, airline policies
Episodic Memory	Checkpoints / Conversation History	Previous booking sessions, past decisions, retry history
Procedural Memory	System Prompts of Worker Agents	"You are a flight specialist. Never book without confirmation."

← Ch. 02: Perception, Memory, Planning & Learning introduced these types as cognitive layers.

PART 3

Error Handling & Replanning

When agents fail — and how the system recovers



Failure Modes in Agentic Systems

Errors are not exceptions — they are expected states. Design for them from the beginning.

Failure Type	Example	Symptom
Tool Error	API responds with 503	Worker delivers no result
LLM Output Error	Output doesn't match expected format	JSON parse failure in Structured Output
Logic Error	Worker books hotel before flight time confirmed	Inconsistent state
Timeout	Worker doesn't respond within defined steps	Orchestrator waits indefinitely
Cascading Failure	Flight Worker fails → Hotel Worker uses wrong premise	System-wide inconsistent state

← Ch. 04 Slide 19: deadline: 'respond_within_steps: 3' in the Agent Message anatomy was already preparing for timeout handling.

Three Recovery Strategies

Choose the strategy based on the failure type and the criticality of the missing result.

1



Retry with Backoff

When: Transient errors: network blips, API rate limits, temporary unavailability. **How:** Call the same tool again with increasing wait times between attempts. Max 2–3 retries. **Limit:** Does not help with logic errors or persistent failures.

2



Graceful Degradation

When: A worker permanently fails and a partial result is acceptable. **How:** Return what's available. Flag what's missing. Let the user decide the next step. **Limit:** Requires the system to define "acceptable partial" upfront.

3



Replanning

When: Orchestrator receives error state and can find an alternative path. **How:** Orchestrator reads full state, identifies the failure, reformulates the plan, re-delegates. **Limit:** Requires typed shared state and checkpoint support — Part 2 of this chapter.

Retry with Backoff – Implementation

Retry logic lives inside the Worker node function. The State carries the error for the Orchestrator to read.

```
import time

def flight_worker(state: TravelState) -> TravelState:
    """
    Worker node with retry + backoff.
    Max 3 attempts, exponential wait.
    """
    for attempt in range(3):
        try:
            result = search_flights(state["user_request"])
            return {**state, "flights": result}

        except APIError as e:
            if attempt == 2:
                errors = state["errors"] + [f"Flight failed: {e}"]
                return {**state, "errors": errors}
            time.sleep(2 ** attempt)  # 1s, 2s, 4s
```

Return updated state

Always return the full state dict with new or changed fields. Never return only a partial update.

Exponential backoff

Attempt 0: 1s wait, Attempt 1: 2s wait, Attempt 2: 4s wait. Prevents hammering a struggling service.

Error enters the state

The error is written into state["errors"]. The Orchestrator reads it and decides: degrade or replan.

Errors Are Expected State

Design principle: failure paths deserve the same architectural attention as success paths.

✗ Without Error State Design

Errors crash the agent pipeline

No error field in state — failures are silent

Orchestrator has no info on what failed

System must restart from scratch on any failure

✓ With Error State Design

Errors are caught and written to state["errors"]

Explicit error field: always present, sometimes empty

Orchestrator reads error state, decides: retry / degrade / replan

Checkpoints allow resuming from last valid state

→ Ch. 06: Unhandled error states are also a security risk — a failed agent may leave the system in an exploitable intermediate state.

PART 4

Testing Non-Deterministic Systems

When assertEquals no longer works — and what replaces it



The assertEquals Problem

Classical unit tests assume determinism. LLM-based agents are inherently non-deterministic.

```
# This test FAILS on second run ❌
def test_flight_result():
    result = run_agent("Book HAM→NRT")
    assert result["flights"] == [
        {"flight": "LH730", "price": 890}
    ]

# ← same input, different output every run!
```

Why non-determinism?

- **Temperature & sampling:** LLMs produce probabilistic outputs.
- **Context sensitivity:** Tiny context differences shift outputs.
- **Tool result variance:** APIs return different data over time.
- **Multi-agent interaction:** Worker outputs influence each other.

What to do instead:

Semantic Assertions

Check structure and meaning, not exact value. Does the output contain a flight date? Is it in the future?

Behavioral Checks

Was the right tool called? Was an error correctly handled? Did the agent request human approval?

Golden Dataset + Similarity

Compare against known-good outputs using similarity metrics or LLM-as-Judge.

Tracing

Log every step. When something goes wrong, inspect the full trace — not just the output.

Four Testing Strategies

Use all four in combination. Each catches a different class of failure.



Semantic Assertions

Check meaning and structure, not exact string equality.

```
result['flights'] is not None
all(f['date'] for f in result['flights'])
'departure' in result['itinerary']
```

When: Always — minimal baseline for every test.



Behavioral Testing

Check what the system did — not what it returned.

```
Did search_flights() get called?
Did retry fire after 503?
Did orchestrator request human approval on
uncertainty?
```

When: Testing invariants: behaviors that must always happen.



Golden Dataset

Curated test cases with known-good reference outputs.

```
Run agent on 20 curated inputs. Compare output to
reference via cosine similarity or LLM judge.
Track pass rate over time.
```

When: Regression testing after model or prompt changes.



Tracing

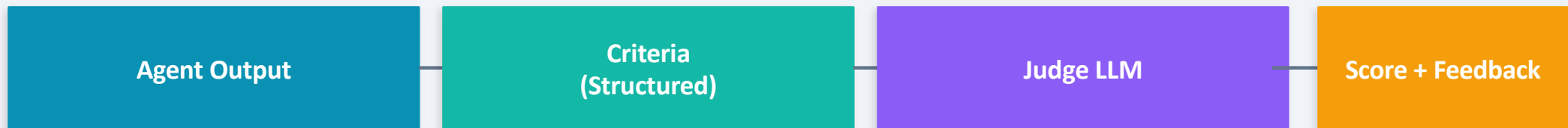
Full recording of all steps: nodes, state transitions, LLM calls, tool calls, intermediate results.

```
Use LangSmith, Arize Phoenix, or W&B Weave.
Inspect traces after unexpected outputs.
```

When: Debugging, production monitoring, audit.

LLM-as-Judge

Use a second LLM to evaluate the output of the first — on structured, defined criteria.



✓ Advantages

- Handles outputs too complex for rule-based checks
- Evaluates nuance: tone, completeness, coherence
- Scalable: evaluate hundreds of test cases automatically
- Can provide explanations, not just pass/fail scores

⚠ Limitations

- Not ground truth — the judge can be wrong too
- Criteria must be specific and measurable
- Adds latency and cost to the test suite
- Works best as complement to semantic assertions, not a replacement

Tracing Tools Overview

Observability is not a feature to add later — plan for tracing from the first line of code.



LangSmith

LangChain / LangGraph

Strengths:

- Native LangGraph integration
- Visual trace tree per run
- Side-by-side comparison of runs
- Evaluation pipelines built-in

Best for: Teams already using LangGraph/LangChain.



Arize Phoenix

Arize AI (open-source)

Strengths:

- Framework-agnostic
- LLM span visualization
- Embedding drift detection
- Self-hostable

Best for: Teams needing full control and open-source stack.



W&B Weave

Weights & Biases

Strengths:

- Integrates with W&B experiment tracking
- Automatic call logging
- Evaluation dataset management
- Works across frameworks

Best for: Teams with existing W&B ML experiment tracking.

Observability as Architecture

A production-ready agentic system is always observable. Without tracing, there are no guardrails.

Core Principle

Without observability, an agentic system is a black box. You cannot debug what you cannot see. You cannot guard what you cannot trace. Tracing is not an optional enhancement — it is a prerequisite for both production readiness and safety.



Tracing

Every node execution, every state transition, every LLM call is logged with inputs and outputs.



State Inspection

The complete state dict is readable at any point during and after execution.



Alerting

Thresholds trigger on error rates, latency, unexpected tool calls — before users notice failures.

→ Ch. 06: Missing observability is not just an engineering problem — it is a safety risk. Without traces, no guardrail can be defined or enforced.

What tracing reveals: Which tools were called and in which order? · What was the exact LLM prompt and response at each step? · Which state transitions happened — and why? · Where did the system spend most of its time / tokens?

Key Takeaways

1

The framework makes architecture decisions.

LangGraph, AutoGen, CrewAI have different opinions on state and communication. Choose deliberately.

2

State management is not a detail.

Unmodeled state = implicit state = consistency bugs. Always use Shared State for multi-agent workflows.

3

Checkpoints enable replanning.

A system without checkpoints must restart on failure. With checkpoints, the orchestrator replans from the last valid snapshot.

4

Test behavior, not output equality.

Semantic assertions, behavioral checks, golden datasets, and tracing replace classical assertEquals in non-deterministic systems.

5

Observability is architecture.

An agentic system without tracing is not production-ready — and not safe. Plan for observability from the first line.

Safety & Guardrails

We can now build a multi-agent system. But what happens if an agent is deliberately misdirected?

What is Prompt Injection in a Multi-Agent System?

An attacker injects instructions into a worker's output — and the orchestrator executes them.

How do you prevent an agent from doing more than it is allowed?

Guardrails, scope restrictions, Human-in-the-Loop — and when is each sufficient?

Who is liable when a multi-agent system makes a wrong booking?

The question of accountability for autonomous systems — no trivial answer.

References & Further Reading

LangGraph Documentation (2024): langgraph.dev – Official docs, tutorials, examples

Wu, Q. et al. (2023): "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation" – Microsoft Research

Hong, S. et al. (2023): "MetaGPT: Meta Programming for Multi-Agent Collaborative Framework" – arXiv

Zaharia, M. et al. (2024): "The Shift from Models to Compound AI Systems" – BAIR Blog, Berkeley

Shankar, S. et al. (2024): "Who Validates the Validators? Aligning LLM-Assisted Evaluation of LLM Outputs" – arXiv

Arize Phoenix (open-source): Tracing and evaluation for LLM applications – arize.com/open-source

Weights & Biases Weave: LLM observability and evaluation – wandb.ai/weave